

Measuring Validator Utility in Generate–Verify–Repair NetOps Pipelines: a Confirmatory Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory study (AC-2026-03) to evaluate whether a multidimensional validator metric suite predicts end-to-end agentic NetOps success better than a single verifier pass rate. Using a synthetic intent→configuration generator and a hosted generate–validate–repair (GVR) adapter under shared_initial_candidate semantics, we ran 200 paired tasks (one shared initial generation per task reused for baseline and guarded). The deterministic validator (deterministic_netops_validator_v5, v5.0) judged every sampled initial candidate valid; no repairs were applied. Baseline = 200/200, guarded = 200/200, paired difference = 0.00 (bootstrap 95% CI [0.00, 0.00], exact McNemar two-sided $p = 1.0$; bootstrap_seed = 1729, resamples = 10000). The observed ceiling invalidated the preregistered power assumptions (targeted 10 percentage-point paired improvement) and rendered the primary confirmatory test uninformative for the originally planned effect. We report preregistered design choices, precise execution accounting, representative validator-trace excerpts, quantitative analysis, failure-mode diagnostics, and artifact-grounded prescriptions for follow-up preregistered experiments (fault injection, multi-sample generation, multi-validator comparison) required to estimate validator coverage and repair value.

I. INTRODUCTION

Automating NetOps workflows with generative models requires reliable mechanisms to avoid harmful, silent, or wrong-but-plausible actions. A common operational pattern is generate–verify–repair (GVR): a generator (LLM) proposes a candidate configuration or plan, a deterministic validator mechanically checks correctness and policy constraints, and, when necessary, a repair module synthesizes corrected candidates or triggers human review. Prior demonstrations of GVR-like architectures in code and systems motivate their adoption in NetOps, but systematic, preregistered measurements of validator coverage (false-positive/false-negative rates), detection latency, and predictive usefulness for end-to-end guarded success are scarce [1].

We preregistered study AC-2026-03 with a paired design that isolates the effect of downstream validator-triggered repair under shared_initial_candidate semantics: for each task the generator produced a single initial candidate that was reused by both the baseline (no repair) and guarded (validator+repair permitted) conditions. Under these semantics any paired difference can only arise down-

stream from validator-triggered repair, not from independent generator sampling. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline and the preregistered confirmatory hypothesis (H1) expected that a multidimensional validator-metric suite would explain more variance in end-to-end success than a single verifier pass rate. Our main contribution is a fully preregistered, artifact-archived experiment and an exact, transparent report of a degenerate confirmatory outcome (ceiling) with concrete, artifact-grounded prescriptions for follow-up designs that can measure validator utility.

II. RELATED WORK

Work across code, systems, and NetOps communities has proposed and prototyped generate–verify–repair or closed-loop verifier architectures to reduce stochastic generator error and enable deterministic acceptance criteria [1]. Recent NetOps and AIOps benchmark efforts document substantive operational failure modes (ticket noise, false premises, and wrong-but-plausible outputs) that complicate end-to-end automation and motivate insertion of verifier layers [2],[3]. Independent system reports and preprints propose self-healing orchestrators, auditable decision traces, and verifier-guided repair to reduce silent or action-triggering failures in tool-augmented LLM systems [4]. Surveys of LLMs for telecommunications and NetOps discuss adaptation and verification trade-offs and motivate metricized evaluation of verification cost versus nominal correctness [5].

These verified records motivate a measurement focus on validator soundness and operational impact but do not provide a standardized, empirically estimable validator-metric suite tailored to NetOps GVR flows. The present preregistered experiment was designed to begin filling that measurement gap under strict preregistration and archived-execution practices; when interpreted against the cited literature we treat prior demonstrations as architectural and motivational rather than as establishing empirical coverage for NetOps validators.

III. METHODOLOGY

Preregistration and primary design. The study preregistration (AC-2026-03) fixed a paired confirmatory de-

sign executed via the `hosted_netops_gvr_v1` adapter family in `scientific_confirmatory` mode. The design choice `generation_semantics = shared_initial_candidate` and `initial_generation_calls_per_task = 1` was central: for each deterministic task index the adapter made exactly one initial generator call and that identical generated candidate (the shared initial candidate) was provided to both conditions in the pair. This preregistered semantics guarantees that any observed paired difference (guarded minus baseline) can only arise from downstream deterministic validator behaviour and any permitted repair, not from independent generator sampling differences. The preregistered primary estimand was the `paired_success_rate_difference_guarded_minus_baseline`. The confirmatory test specified an exact two-sided McNemar test for paired binary outcomes, supplemented by paired bootstrap percentile confidence intervals (`bootstrap_resamples = 10000`, `bootstrap_seed = 1729`) as recorded in the analysis artifacts.

Model, sampling, and execution accounting. The declared model in `execution/model_configuration.json` is "gpt-5-mini" (`requested_model = gpt-5-mini`). The recorded model configuration also contains operational fields: `max_output_tokens = 2000`, `maximum_attempts_per_call = 1`, `provider = "openai_responses"`, `store = false`, and `reasoning_effort = "minimal"`. Crucially, `execution/model_configuration.json` records `temperature = null` (temperature unset). Null/unset is not evidence of deterministic provider-side sampling and does not imply `temperature = 0`; runtime provider sampling behavior may still be stochastic and is audited via the provider-call records. The execution manifest and `deterministic_reconciliation` record provider-call audit fields consistent with the preregistered single shared generation per task: `hosted_model_call_event_count = 200`, `completed_hosted_model_call_event_count = 200`, `cache_hit_count = 0`, `cache_miss_count = 200`, and `stage_counts.shared_initial_generation = 200`. Because the preregistered model-call budget permitted up to `maximum_model_calls = 400`, the actual `model_calls_used = 200` demonstrates strict adherence to the shared-initial single-sample policy (one hosted-model invocation per task index).

Task generation, constraints, and contamination controls. Tasks were drawn deterministically from `deterministic_netops_task_generator_v5` with `task_family = intent_configuration_repair_v1` and the preregistered deterministic index-rule `challenging_workflow_stress_v1`. Each task manifest encodes: `initial_state`, `expected_final_state`, an explicit natural-language intent, `allowed_fields` and `allowed_touched_fields`, `workflow_policies` (for example, `minimum_active_in_groups`, `must_be_down_for_fields`, `protected_interfaces`), a `required_command_sequence`, and a structured difficulty label ("very_hard"/"extreme") that the generator used when instantiating the reference answer and task inputs. To preserve confirmatory integrity the preregistration specified deterministic contamination/exclusion rules: exact-duplicate outputs (identical SHA-256) and near-duplicate tasks (embedding cosine similarity > 0.95) were to

be detected automatically and replaced deterministically by the next-index entry to maintain the planned `task_count`. The analysis artifacts show no contamination exclusions occurred (`analysis/contamination_summary.csv` empty) and the missingness summary reports `complete_pairs = 200`, consistent with the preregistered sampling plan.

Validator, scoring rule, and permitted repair policy. The deterministic validator used throughout the study was `deterministic_netops_validator_v5` (`validator_version = 5.0`). The validator implements mechanistic checks that are directly exposed in per-episode `validator_trace` fields archived for each episode: `normalized_configuration` (canonical command list), `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_count` and `violation_codes`, `valid` (boolean), `validator_id/version`, `repair_applied` (boolean), and (when applicable) `repair_provider_trace` metadata. The preregistered binary scoring rule for the confirmatory estimand was `full_intent_constraint_validation`, which maps directly to `validator.valid` (`true = success`, `false = failure`) in the archived episodes. The pipeline permitted at most one repair call per episode when the validator rejected a candidate; repair events would set `repair_applied = true` and populate `repair_provider_trace` fields (these were null for all episodes in this execution). The preregistration also deterministically specified that any repair introducing actions outside the task's `allowed_action` set would be treated as a policy-violation repair and excluded from the primary success numerator, with that event logged as a policy-violation.

Execution policy, missingness, and sensitivity planning. The preregistered `maximum_attempts_per_call = 1` disallowed manual retries; adapter and provider failures were logged and would lead to deterministic exclusions or to classification as technical failures per the missingness plan. Primary confirmatory analysis used only complete pairs; the preregistered sensitivity analysis treated any missing/incomplete episode as a failure (0) for guarded episodes to produce conservative bounds. The execution manifest records `completed_episode_count = 400`, `failed_episode_count = 0`, and `complete_pair_count = 200`; the analysis missingness artifact confirms no observed missing pairs (`analysis/missingness_summary.csv`). All missingness and audit logs (timestamps, HTTP codes, provider error messages) were archived for reproducibility.

Analysis executor and inferential details. Analysis used the `paired_binary_analysis_v1` executor as preregistered. The executor computed the paired 2×2 contingency counts (`n_11`, `n_10`, `n_01`, `n_00`) from per-episode `validator.valid` labels; primary testing used the exact McNemar two-sided test appropriate for paired binary outcomes. Confidence intervals for the paired difference were constructed using a paired bootstrap percentile procedure with `bootstrap_resamples = 10000` and `bootstrap_seed = 1729` (these bootstrap parameters are recorded in `analysis/analysis_log.jsonl`). Secondary analyses that were preregistered but exploratory (for example, linear models predicting simulated operational-impact proxies from validator metric vectors) were to be treated as exploratory;

multiplicity control was only to be applied to exploratory p-values where reported (Benjamini–Hochberg), while point estimates and intervals were to be reported without multiplicity adjustment otherwise.

Deterministic reconciliation and provider audit semantics. The execution bundle includes deterministic_reconciliation artefacts that reconcile task and provider-call accounting with analysis inputs; this reconciliation confirms that both baseline and guarded conditions observed the same shared_initial_candidate for each pair (cross_condition_cache_key_reuse_observed = false indicates no hidden cross-condition cache reuse beyond the single sampled generation). Provider-call audit fields (hosted_model_call_event_count = 200, cache_miss_count = 200) plus the model_calls_used = 200 entry make explicit that one model call produced the shared_initial_candidate per task. These audit fields are central for verifying that the experimental estimand (paired difference under shared_initial_candidate) is correctly identified in the archived artifacts.

Reproducibility, archival artifacts, and permitted reanalyses. All raw responses, per-episode validator traces, provider-call traces, task manifests, model configuration JSON, analysis code, and final numeric artifacts (paired_contingency_table.csv, condition_summary.csv, analysis logs) are archived in the supplied execution bundle referenced elsewhere in this paper. The preregistered artifact set supports direct reanalysis under alternative scoring rules (for example, judging success against a looser parse-based rule), simulated fault-injection applied offline to archived reference answers, or a multi-sample re-run using the same task indices but altering initial_generation_calls_per_task; such follow-up analyses are precisely the artifact-grounded prescriptions we propose and can be preregistered and executed using the same adapter semantics. Important operational note recorded in execution/model_configuration.json: the stored temperature field is null/unset; null does not imply deterministic sampling by hosted providers and therefore does not remove the need to audit provider-call records when assessing generator variance.

IV. RESULTS

Execution accounting and raw outcome summary. The run completed exactly per preregistration: planned_episode_count = 400, completed_episode_count = 400, failed_episode_count = 0, complete_pair_count = 200, and model_calls_used = 200 (one shared initial generation per pair). Provider-call audit entries show hosted_model_call_event_count = 200, completed_hosted_model_call_event_count = 200, cache_hit_count = 0, and cache_miss_count = 200. The analysis condition summary (analysis/condition_summary.csv) reports: baseline successes = 200, baseline failures = 0 (success_rate = 1.0); guarded successes = 200, guarded failures = 0 (success_rate = 1.0).

Primary confirmatory statistics and exact contingency. The paired contingency (analysis/paired_contingency_table.csv) is

$n_{11} = 200$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$; that is, every pair was valid under both baseline and guarded. The paired_success_rate_difference_guarded_minus_baseline equals 0.00. The preregistered exact McNemar two-sided test returned $p = 1.0$ because there are zero discordant pairs ($n_{10} + n_{01} = 0$). The preregistered bootstrap-percentile confidence interval (bootstrap_resamples = 10000, bootstrap_seed = 1729) is [0.00, 0.00], reflecting a point mass at zero in the resampled paired-difference distribution. These analysis outputs are recorded verbatim in analysis artifacts (analysis/analysis_log.jsonl, analysis/paired_contingency_table.csv).

Representative validator-trace and why no repairs occurred. A typical archived validator_trace (pair-000004) exhibits the mechanistic checks the validator enforces and illustrates why no repair was triggered:

```
pair_id: pair-000004 normalized_configuration: "interface
edge2 admin down\ninterface edge2 vlan 40\ninterface
edge2 admin up\ninterface edge1 admin down"
parsed_command_count: 4 required_command_count: 4
observed_touched_field_count: 3 valid: true repair_applied:
false validator_id: deterministic_netops_validator_v5
validator_version: 5.0
```

Equivalent validator_trace.validation_after.valid = true and repair_applied = false hold for every archived episode; repair_provider_trace fields are null for all episodes. Representative task manifests show required_command_count equal to parsed_command_count for the accepted candidates, indicating structural completeness under the validator’s rules. Because baseline and guarded reused the identical shared_initial_candidate per pair, these per-episode validator outputs explain the degenerate paired outcome: there were no validator-detected mechanical defects to correct.

Expanded diagnostics: what the artifacts reveal about failure modes and what remains unobserved. Several archived artifacts together explain the ceiling outcome without invoking unrecorded assumptions: - deterministic_reconciliation and model_configuration.json show the single-shared-generation accounting (one initial draw per task; model_calls_used = 200). This sampling regime eliminated generator-sampling variance as a potential source of discordant pairs. - analysis/contamination_summary.csv and analysis/missingness_summary.csv both report no contamination flags and complete_pairs = 200, confirming the confirmatory dataset was contamination-free and complete as preregistered. - validator traces uniformly report valid = true and repair_applied = false; thus per-validator FP/FN estimates are degenerate (no false positives and no false negatives observed relative to the validator’s own criteria on this sampled set), and detection-latency/repair-invocation distributions are uninformative because no repair events occurred.

Power and inferential consequences in artifact terms. The preregistered power calculation targeted detection of an absolute paired improvement of 0.10 with $\alpha = 0.05$ and $\approx 80\%$ power under conservative assumptions (see preregistration). Those calculations assumed non-degenerate baseline variability (for example baseline p values in {0.3,0.4,0.5,0.6,0.7}).

The artifact-grounded observed `baseline_success_rate` = 1.0 (200/200) makes the preregistered detectable effect logically unattainable in this execution: with no observed failures in baseline there are no discordant baseline-only episodes to be corrected by the guarded condition, so the paired estimator cannot register the preregistered 10-percentage-point improvement. We therefore report the confirmatory null outcome as an artifact-valid, design-driven ceiling rather than as evidence that validators lack operational value generally.

What the archived traces allow and what they do not. The execution bundle contains full per-episode normalized configurations, `required_command_count` and `parsed_command_count` fields, provider-call traces, and analysis CSVs. These artifacts enable reproducible reanalyses under alternative scoring rules (for example, scoring that treats specific semantic mismatches as failures) or under changed sampling semantics (multi-sample per task) or injected faults. What the current artifacts do not support is estimation of repair effectiveness or validator false-negative rates in unperturbed production-like inputs because `repair_applied` = false for every episode and no injected faults were executed in this run.

Concluding diagnostic summary. The degenerate all-valid outcome arises from the conjunction of (1) a deterministic task generator that produced candidates aligned with the validator’s mechanistic constraints for the sampled seeds, (2) a single-sample-per-task `shared_initial_candidate` design that removed generator variability as a potential source of discordance, and (3) a deterministic validator that enforces structural and policy constraints but does not attempt to model operator intent beyond the encoded `workflow_policies`. These artifact-grounded determinants fully account for the lack of observed repairs and the uninformative confirmatory statistic; follow-up preregistered arms (fault injection, multi-sample generation, multi-validator comparison) are required — and are supported by the archived infrastructure — to produce non-degenerate estimates of validator coverage, FP/FN rates, repair invocation utility, and detection latency.

V. DISCUSSION

Design implications of `shared_initial_candidate` semantics. The preregistered `shared_initial_candidate` choice isolates downstream validator effects by ensuring both conditions evaluate the same initial candidate for each pair. This isolation is scientifically valuable because any observed paired difference must derive from validator-triggered repair. It also concentrates the risk of a single-sample ceiling: if the generator sample aligns with the validator for all tasks, the design yields zero observed variance in the primary estimand, as occurred here. We emphasize this property because it determines what the paired estimator can and cannot measure: it cannot detect differences attributable to generator sampling or to alternative prompt constraints when the sampling semantics were shared.

Operational interpretation and validator scope. The deterministic validator (`deterministic_netops_validator_v5`) reliably enforces mechanistic intent-constraint checks

(`parsed_command_count`, `required_command_count`, `violation_codes`, `valid` flag). However, artifact-grounded evidence here shows that correctness-by-structure does not imply semantic intent alignment under operator expectations. That limitation is intrinsic to validators that lack an external intent model or operator feedback; detecting semantic mismatches requires either richer intent models, human-in-the-loop signals, or tailored semantic validators. The present execution does not imply validators are unnecessary—rather, it demonstrates that in a synthetic bank and single-sample regime the validator had nothing mechanical to correct.

Expanded diagnostics grounded in execution artifacts. The archived execution provides explicit numeric diagnostics that clarify why the confirmatory estimator was uninformative: `model_calls_used` = 200 (one shared generation per pair), `complete_pair_count` = 200, `baseline_success_count` = 200, `guarded_success_count` = 200, and `repair_applied` = false observed for every episode. The paired contingency is a point mass (n_{11} = 200), and the bootstrap procedure (`bootstrap_seed` = 1729, `resamples` = 10000) produced a degenerate sampling distribution with 95% percentile CI [0.00, 0.00]. These exact values are available in the analysis artifacts (condition summary and paired contingency outputs) and show that the confirmatory null is not a statistical fluke but a deterministic consequence of the joint generator-validator-sampling configuration used.

Practical lessons for preregistered validator evaluation. From a methodological perspective, two trade-offs are clear and artifact-evident. First, `shared_initial_candidate` is a powerful control: it guarantees that any observed paired improvement is attributable to downstream validator/repair logic alone. Second, that control increases sensitivity to ceiling effects when generation produces validator-compliant candidates for the chosen seeds. For studies whose primary aim is to measure validator coverage (FP/FN) and repair utility, the preregistered design should therefore include one or both of the following preregistered modifications to avoid degeneracy: (a) a controlled fault-injection arm that deterministically introduces a known fraction f of invalid cases (syntactic, configuration-logic, or semantic-intent swaps) into the evaluation set so that the validator will face realistic defects; (b) a multi-sample generation arm with `initial_generation_calls_per_task` ≥ 3 independent draws per task to reintroduce generator variability and produce candidate diversity that can trigger validator rejections and repairs. Both prescriptions were articulated in the archived post-run prescriptions and are directly implementable within the same adapter semantics.

What validator metrics become estimable under those modifications. If faults are injected or multiple independent draws are allowed, the validator-metric suite preregistered in AC-2026-03 becomes estimable: (i) per-validator false-positive rate (FP-rate) as the fraction of `validator.accept` = true when the task’s ground-truth is invalid; (ii) per-validator false-negative rate (FN-rate) as `validator.reject` = false for candidates that do actually violate the required intent sequence; (iii) mean detection latency measured from model-response receipt to

validator decision (records include timestamps in provider- and validator-traces); (iv) repair-invocation rate and post-repair success fraction when `repair_applied = true` is observed. These metrics map directly to the fields already emitted by the validator in the archived traces (`valid`, `violation_count`, `repair_applied`, `parsed_command_count`) and to provider-call audit counters; no additional opaque instrumentation is required to compute them once non-degenerate data are present.

Reproducibility and artifact-grounded protocol refinements. The execution artifacts document concrete accounting fields that support exact replication of the intended follow-ups: provider-call audit counts (`hosted_model_call_event_count = 200`), the model identifier (`declared_model_version = gpt-5-mini`), the recorded temperature = null/unset (explicitly not evidence of deterministic sampling), and the validator identifier/version (`deterministic_netops_validator_v5, v5.0`). A follow-up preregistration should (and can, per the archived infrastructure) explicitly set: (1) `initial_generation_calls_per_task = k` (e.g., $k = 3$) with independent generation seeds recorded for each draw; (2) a deterministic fault-injection schedule specifying fault classes and the exact fraction f of tasks affected; and (3) a plan to measure detection latency using the existing timestamps. These concrete protocol elements are already supported by the adapter and manifest fields and would remove ambiguity about sampling and allow FP/FN estimation without inventing new instrumentation.

Threats to validity revisited with artifact evidence. Internal validity: the `shared_initial_candidate` design intentionally removes generator sampling as a confound but introduces a ceiling risk; the exact accounting shows that risk materialized (complete agreement across conditions). Construct validity: `validator.valid` captures mechanistic constraint satisfaction but not semantic intent alignment—this remains the central construct gap and explains why correct-by-structure outputs can still be operationally unacceptable. External validity: the study executed one model (`gpt-5-mini`) and one deterministic validator against a synthetic task bank; therefore caution is required when extrapolating to multi-vendor production networks or to models and validators with different coverage. Conclusion validity: the archived bootstrap and contingency outputs (seed/resamples specified) demonstrate the degenerate inferential state (zero variance) and justify treating the confirmatory hypothesis as untestable under these run parameters rather than as a failed hypothesis test.

Operational implications and recommended reporting practices. For experimenters and operators evaluating GVR pipelines, we recommend preregistering both (a) the sampling semantics (`shared_initial_candidate` vs independent draws) and (b) explicit anti-ceiling design elements (fault injection or multi-draw sampling) whenever the evaluation goal is to estimate validator coverage or repair utility. Empirically reporting the exact counts we used (`model_calls_used`, `complete_pair_count`, baseline/guarded success counts, `repair_applied` counts, bootstrap_seed/resamples) increases interpretability because readers can immediately diagnose whether

a noninformative null arose from lack of validator failures or from insufficient sample diversity.

Summary. The execution artifacts and numeric diagnostics make the outcome clear: under the chosen `shared_initial_candidate` semantics, the combination of deterministic seeds, single-sample generation, and a mechanistic validator that checks structural constraints produced a ceiling effect (200/200 valid). That outcome is scientifically informative about the joint behavior of generator+validator under these settings and usefully constrains what conclusions can and cannot be drawn. It also supplies a concrete, artifact-grounded roadmap—fault injection, multi-sample generation, and multi-validator comparison—that will enable future preregistered studies to estimate the validator metrics of interest and to quantify repair utility in operationally meaningful conditions.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics constrained inference: baseline and guarded reused the same initial candidate per pair; any paired difference could only arise from validator-triggered repair (not from independent generator sampling).
- Synthetic task bank (difficulty 'very_hard'/'extreme') is not equivalent to heterogeneous production networks (multi-vendor devices, real ticket noise); external validity is limited.
- Single-model experiment: only `gpt-5-mini` was executed; no multi-model generality claims are made.
- No repairs were applied because all initial candidates passed the deterministic validator; therefore the study could not estimate repair effectiveness or validator false-negative behavior in this execution.
- Validator scope: `deterministic_netops_validator_v5` enforces mechanistic intent-constraint checks but does not guarantee detection of semantic intent misalignment (correct-by-structure but incorrect-by-intent).
- Recorded execution/`model_configuration.json` temperature = null/unset does not imply deterministic sampling; provider-side sampling behavior may vary and is documented in execution manifests.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory study (AC-2026-03) under `shared_initial_candidate` semantics to measure validator utility in NetOps GVR pipelines. For the synthetic task bank, the sampled `gpt-5-mini` generations, and `deterministic_netops_validator_v5` used here, every sampled initial candidate passed the validator's mechanistic checks so no repairs were applied and guarded success equaled baseline (200/200). The preregistered power assumptions (targeted 10-percentage-point improvement) were invalidated by this ceiling outcome. The result is artifact-grounded and reproducible from the archived bundle. We publish the exact execution and analysis artifacts to support replication and provide concrete, preregistered experiment prescriptions (fault injection, multi-sample generation, multi-validator comparison) that are

required to produce estimable validator FP/FN behavior and repair value in operationally meaningful conditions.

DISCLOSURE STATEMENT

Disclosure (concise): Declared model and provider: gpt-5-mini via provider bundle 'openai_responses'. Orchestration/adaptor: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5 (validator_version = 5.0). Preregistration: AC-2026-03 (preregistration_sha256 = de37b485421cb0895a98df38b6b3baad1dc7c13c81065e3d8240d2a94a0ff844). Master prompt immutable reference: provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba. Autonomy boundary: a human Research Director selected the broad topic family and approved the master prompt before the autonomy lock; after the lock the AI system autonomously performed literature review, preregistration, experiment design, execution, analysis, internal peer review and manuscript drafting without manual editing of technical content. Sampling/generation semantics: shared_initial_candidate (one initial sampled generation per task reused for baseline and guarded); execution/model_configuration.json recorded temperature = null/unset (null/unset is not evidence of deterministic sampling and does not imply temperature = 0). Call budget and usage (preregistered): maximum_model_calls = 400; actual model_calls_used = 200 (one shared initial generation per pair). All raw outputs, per-episode validator traces, execution manifests, and analysis artifacts are archived in the supplied execution bundle (see execution/execution_manifest.json). No human manually edited the manuscript technical content after the autonomy lock.

REFERENCES

- [1] [1] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN Electronic Journal, 2026. doi:10.2139/ssrn.6754899.
- [2] [2] K.-H. Tseng, N. Bogahawatta, Y. Ginige, K. Patel, K. Dacic, S. Seneviratne, "Fault-Bench: Towards Benchmarking Network Troubleshooting LLM Agents under Unreliable User Tickets," arXiv:2608.27021, 2026.
- [3] [3] Y. Liu, C. Pei, L. Xu, B. Chen, M. Sun, Z. Zhang, Y. Sun, S. Zhang, K. Wang, H. Zhang, J. Li, G. Xie, X. Wen, X. Nie, M. Ma, P. Dan, "OpsEval: A Comprehensive IT Operations Benchmark Suite for Large Language Models," arXiv:2310.07637, 2023.
- [4] [4] R. S. Babu and A. Agrawal, "Self-Healing Agentic Orchestrators for Reliable Tool-Augmented Large Language Model Systems," arXiv:2606.01416, 2026.
- [5] [5] H. Zhou, C. Hu, Y. Yuan, Y. Cui, Y. Jin, C. Chen, H. Wu, D. Yuan, L. Jiang, D. Wu, X. Liu, J. Zhang, X. Wang, J. Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities," IEEE Commun. Surveys & Tutorials, 2024. doi:10.1109/COMST.2024.3465447.